

A Medieval Field Cannon in OpenGL

Hierarchical Transformations, Rolling Constraints and the Phong Reflection Model

Computer Graphics Project Report

September 5, 2026

Abstract

This report documents a real-time 3-D scene, written in Python with PyOpenGL and freeglut, of a medieval field cannon on a wheeled carriage. The carriage can be driven along the ground with the wheels rolling at exactly the right rate, the barrel elevates about its trunnions, recoils along its own axis as a damped spring when fired, and launches a cannonball that follows a ballistic arc. The whole assembly is built from analytic primitives whose surface normals are derived rather than approximated, and is lit by a single movable point light using the full Phong reflection model — evaluated *per fragment* in GLSL, with a per-vertex fixed-function path available for direct comparison. Every quantitative claim in this document is reproduced by `verify.py`, whose output is included verbatim in Appendix A, and every figure is a real frame captured from the running program.

Contents

1	Introduction	1
1.1	Requirement coverage	2
2	How to run the project	2
2.1	Prerequisites	2
2.2	Installation	3
2.3	Running	3
2.4	Controls	3
2.5	Files	3
2.6	Troubleshooting	4
3	Architecture	4
4	The mathematics	4
4.1	Homogeneous coordinates and affine transforms	4
4.2	The matrix stack is a scene-graph traversal	5
4.3	Rolling without slipping	6
4.4	Rotation about a pivot that is not the origin	7
4.5	Where is the muzzle?	7
4.6	Recoil as a damped harmonic oscillator	8
4.7	Projectile motion	9
4.8	Viewing and projection	10
4.9	Normals	11
4.10	The Phong reflection model	12
4.11	Gouraud versus Phong shading	14
4.12	Atmospheric fog	15
5	Implementation notes	15
5.1	The shader	15
5.2	Drawing unlit geometry inside a lit frame	16
5.3	Geometry budget	16

6	Results	16
7	Limitations and possible extensions	17
A	Verification output	18

1 Introduction

The brief was to build a scene that *visibly* demonstrates a small set of core computer-graphics ideas rather than to build a large one. The cannon is a good vehicle for this because each required concept maps onto a physical part of the object:

- the carriage **translates**, and its wheels must **rotate** at the rate implied by that translation, or the illusion collapses instantly;
- the barrel **rotates about a pivot that is itself carried by the moving carriage**, which is exactly what a hierarchical transformation is for;
- the barrel **translates along its own rotated axis** when it recoils, which cannot be expressed in world coordinates without first composing the parent transforms;
- the wooden carriage and the iron barrel react to light in obviously different ways, which is the clearest possible evidence that a real lighting model is running rather than flat colours.

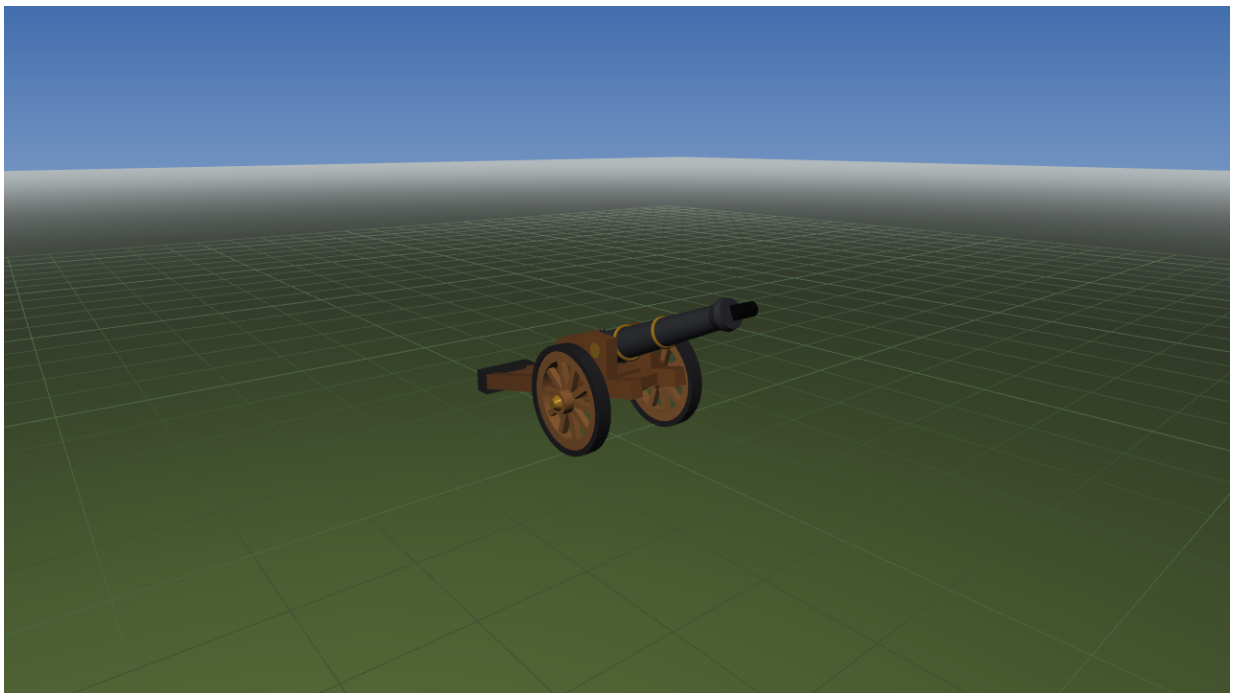


Figure 1: The complete scene. Oak carriage, iron-tired spoked wheels, brass fittings and a cast-iron barrel, lit by one orbiting point light over a 1 m reference grid. Per-fragment Phong shading; exponential-squared fog dissolves the far edge of the ground plane into the sky.

1.1 Requirement coverage

Requirement	Where it is implemented	Evidence
Translation	<code>Cannon.drive</code> , root <code>glTranslatef</code>	Fig. 4
Rotation (rolling)	$\varphi = -s/R$ in <code>Cannon.drive</code>	Fig. 4, §4.3
Rotation (elevation)	<code>glRotatef</code> at the trunnion pivot	Fig. 5
Hierarchical / composite transforms	<code>Cannon.draw</code> push/pop tree	§4.2, Fig. 2
Perspective projection + camera	<code>gluPerspective</code> , <code>gluLookAt</code>	§4.8, Fig. 8
Phong lighting (ambient, diffuse, specular)	<code>lighting.py</code> + GLSL	§4.10, Fig. 12
Two contrasting materials	matte oak vs. shiny cast iron	Fig. 10
Continuous animation / interactivity	recoil spring, projectile, light orbit	Fig. 6, 7

Table 1: Mapping from the assessment criteria to the implementation.

2 How to run the project

2.1 Prerequisites

Component	Requirement	Version used for this report
Python	3.9 or newer	3.14.5 (64-bit, Windows 11)
PyOpenGL	$\geq 3.1.7$	3.1.10
NumPy	≥ 1.24	2.4.4
Pillow	≥ 10.0 (screenshots only)	12.2.0
GPU / driver	OpenGL 2.0+ compatibility profile	AMD Radeon 760M, GL 4.6 compat. (GLSL 4.60)

On Windows the PyOpenGL wheel ships `freeglut64.vc14.dll` inside `OpenGL/DLLS/`, so **no separate freeglut download is needed**. On Linux install `freeglut3-dev` (Debian/Ubuntu: `sudo apt install freeglut3-dev`); on macOS GLUT ships with the system but is deprecated, and the compatibility profile is capped at OpenGL 2.1 — the program still runs, falling back where necessary.

2.2 Installation

```
cd Graphics-project
python -m pip install -r requirements.txt # PyOpenGL, numpy, Pillow
python test.py # sanity check: a coloured triangle
```

If `test.py` shows a red/green/blue triangle, the toolchain is ready.

2.3 Running

```
python main.py # interactive scene
python verify.py # numerical self-checks (no window needed)
python main.py --shots docs/images # re-render every figure in this report, then exit
```

2.4 Controls

Key	Action	Key	Action
← / →	drive backward / forward (wheels roll)	L	light on / off
↑ / ↓	elevate / depress the barrel (0°–45°)	K	light orbit on / off
Space	fire: flash, recoil, cannonball	N	point light ↔ directional
C	cycle the five camera presets	[]	nudge the light azimuth
mouse drag	orbit the camera	F	per-fragment Phong ↔ per-vertex Gouraud
wheel, + / -	zoom	B	Phong ↔ Blinn–Phong specular
W	wireframe	M	cycle the barrel material
G	ground plane on / off	T	projectile trails on / off
J	reference grid on / off	X	clear the projectiles
H	help overlay	U	HUD on / off
R	reset everything	P	save a PNG screenshot
Esc / Q	quit		

Table 2: Complete key bindings. The arrow keys are level-triggered: hold them.

2.5 Files

File	Responsibility
<code>main.py</code>	Window and GL setup, camera, input handling, HUD, main loop, and the <code>-shots</code> driver that renders the report figures.
<code>cannon.py</code>	The <code>Cannon</code> class: dimensions, animation state, the transformation hierarchy, and the numpy mirror of that hierarchy used to locate the muzzle.
<code>lighting.py</code>	<code>Material</code> definitions, the <code>Light</code> rig, the GLSL Phong program, the fog setup and the <code>unlit</code> context manager.
<code>projectile.py</code>	Cannonball integration, ground bounce, fading trail, and the closed-form range predictor used by the HUD.
<code>utils.py</code>	Analytic primitive meshes (box, frustum, tube, sphere, torus, ground) with derived normals, a display-list cache, and 2-D text helpers.
<code>verify.py</code>	Numerical checks of every formula quoted in this report.
<code>test.py</code>	Toolchain sanity check (single triangle).

2.6 Troubleshooting

Symptom	Cause and fix
<code>ModuleNotFoundError: OpenGL</code>	PyOpenGL is not installed for <i>this</i> interpreter. Run <code>python -m pip install PyOpenGL</code> .
<code>OpenGL.error.NullFunctionError: glutInit</code>	freeglut cannot be loaded. On Linux install <code>freeglut3-dev</code> ; on Windows reinstall PyOpenGL so the bundled DLL is present.
Console prints “ <i>GLSL unavailable</i> ”	The driver rejected the shader. The program automatically uses the fixed-function per-vertex path; everything still works, and F simply has no effect.
The window opens but stays black	A remote/software GL context without a depth buffer. Try forcing the vendor GPU, or run with the integrated GPU driver updated.
<code>-shots</code> writes black PNGs	Some drivers refuse to read an occluded back buffer. Keep the window visible and unobstructed while it runs (it takes about two seconds).

3 Architecture

The program is a classic single-threaded GLUT application. `glutIdleFunc` advances the simulation by the measured wall-clock Δt (clamped to 50 ms so a stalled frame cannot teleport the cannon) and requests a redraw; `glutDisplayFunc` renders. One frame is:

1. clear, then paint the three-stop sky gradient in an orthographic pass with depth writes disabled;
2. load the projection (`gluPerspective`) and view (`gluLookAt`) matrices;
3. upload the light position — *after* the view matrix, because OpenGL stores `GL_POSITION` pre-multiplied by the modelview matrix current at the moment of the call, i.e. in eye space;
4. bind the Phong shader (or fall back to fixed-function lighting);
5. draw the ground, the cannon hierarchy, the projectiles, and the light marker;
6. draw the HUD in a second orthographic pass with lighting and fog off.

Every primitive mesh is generated once and compiled into a display list (`utils._cached`), so the per-frame CPU cost is a handful of `glCallList` calls rather than thousands of Python-level `glVertex3f` calls.

4 The mathematics

4.1 Homogeneous coordinates and affine transforms

Points are written $\tilde{\mathbf{P}} = (x, y, z, 1)^\top$ and directions $(x, y, z, 0)^\top$. The trailing 0 is not cosmetic: it makes a direction immune to translation, which is why `Cannon.muzzle_state` can obtain the barrel's axis by pushing $(1, 0, 0, 0)^\top$ through the very same matrix that positions the muzzle tip. The two transforms this project needs are

$$\mathbf{T}(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}, \quad \mathbf{R}_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (1)$$

Both the elevation of the barrel and the spin of the wheels are rotations about $+Z$, because the scene is laid out with $+X$ forward, $+Y$ up and $+Z$ to the right, so Z is the axle direction and also the axis the barrel swings in.

4.2 The matrix stack is a scene-graph traversal

OpenGL's modelview stack turns a tree into code. `glTranslatef` and `glRotatef` *post*-multiply the current matrix, $\mathbf{M} \leftarrow \mathbf{M}\mathbf{A}$, and `glPushMatrix`/`glPopMatrix` save and restore it. A depth-first walk of the tree, pushing on the way down and popping on the way back up, therefore leaves every node with the product of all the transforms on its path from the root — which is precisely the definition of a hierarchical transformation. Reading the code from top to bottom is reading the matrix product from left to right.

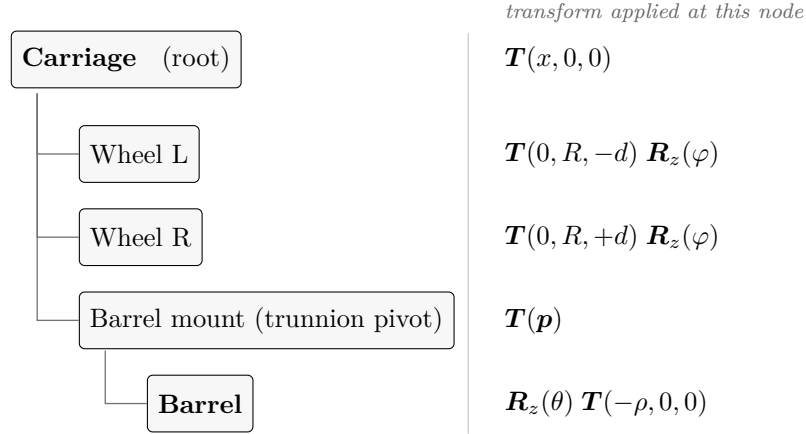


Figure 2: The scene graph. x is the carriage position, φ the wheel angle, $\mathbf{p}$ the trunnion pivot, θ the elevation and ρ the recoil offset. Depth = 2: shallow on purpose.

Writing $\mathbf{V}$ for the view matrix, the composite matrices are

$$\mathbf{M}_{\text{carriage}} = \mathbf{V} \mathbf{T}(x, 0, 0), \quad (2)$$

$$\mathbf{M}_{\text{wheel}} = \mathbf{M}_{\text{carriage}} \mathbf{T}(0, R, \pm d) \mathbf{R}_z(\varphi), \quad (3)$$

$$\mathbf{M}_{\text{mount}} = \mathbf{M}_{\text{carriage}} \mathbf{T}(\mathbf{p}), \quad (4)$$

$$\mathbf{M}_{\text{barrel}} = \mathbf{M}_{\text{mount}} \mathbf{R}_z(\theta) \mathbf{T}(-\rho, 0, 0). \quad (5)$$

The essential observation is that $\mathbf{M}_{\text{barrel}}$ contains $\mathbf{T}(x, 0, 0)$: driving the carriage moves the barrel, the trunnions, the wheels and the muzzle together, for free, because they are all descendants. Check 6 of `verify.py` confirms this numerically — changing only x from 0 to 4 m displaces the muzzle by exactly (4, 0, 0) m while the elevation is held at 25° .

4.3 Rolling without slipping

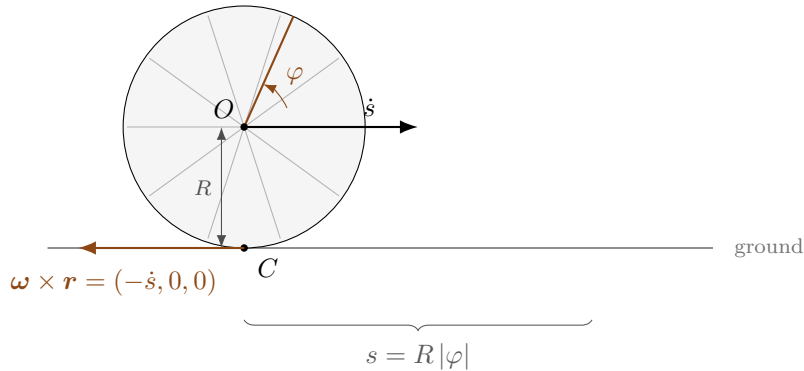


Figure 3: The rolling constraint. The material point at the contact C must be instantaneously at rest, so the spin exactly cancels the translation.

For a wheel of radius R whose centre translates with velocity $\dot{\mathbf{s}} = (\dot{s}, 0, 0)$ and which spins with angular velocity $\boldsymbol{\omega} = (0, 0, \omega_z)$, the velocity of the material point at the ground contact, whose position relative to the centre is $\mathbf{r} = (0, -R, 0)$, is

$$\mathbf{v}_C = \dot{\mathbf{s}} + \boldsymbol{\omega} \times \mathbf{r} = (\dot{s}, 0, 0) + (0, 0, \omega_z) \times (0, -R, 0) = (\dot{s} + \omega_z R, 0, 0). \quad (6)$$

Rolling without slipping means $\mathbf{v}_C = \mathbf{0}$, hence $\omega_z = -\dot{s}/R$, and integrating from rest,

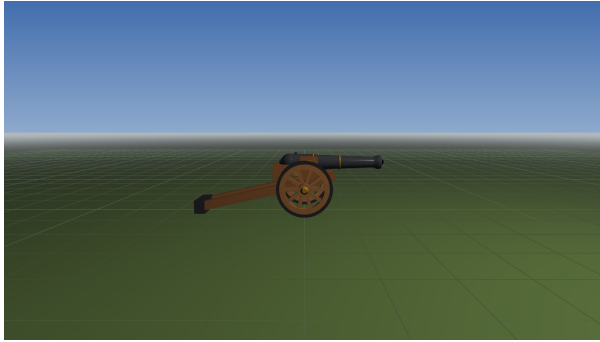
$$\boxed{\varphi = -\frac{s}{R}} \quad (\text{radians; the code stores degrees, } \varphi^\circ = -\frac{180}{\pi} \frac{s}{R}). \quad (7)$$

The minus sign is not decoration: with $+Z$ pointing right, a wheel advancing towards $+X$ turns *clockwise* as seen from $+Z$, which is a negative rotation under the right-hand rule. Get it wrong and the wheels visibly spin backwards, the single most recognisable animation bug there is.

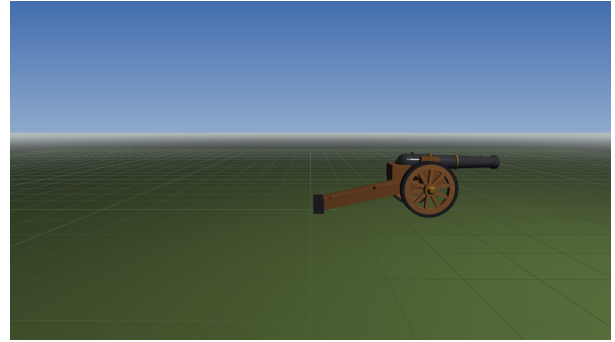
Implementation (`cannon.py`) applies (7) incrementally, using the distance actually moved after clamping so that pushing against the end of the field does not accumulate phantom rotation:

```
new_x = clamp(self.x + ds, X_MIN, X_MAX)
moved = new_x - self.x
self.x = new_x
self.wheel_angle -= math.degrees(moved / WHEEL_RADIUS)
```

Check. Driving $s = 2.6000$ m with $R = 0.6200$ m gives $\varphi = -240.2726^\circ$ analytically and -240.2726° in the simulation (error 2.7×10^{-12} degrees, i.e. floating-point round-off); the rim arc $R|\varphi|$ recovers 2.6000 m. Figure 4 is that exact manoeuvre.



(a) $s = 0$, $\varphi = 0^\circ$



(b) $s = 2.60$ m, $\varphi = -240.27^\circ$

Figure 4: Translation and the rolling constraint, from a camera fixed in *world* space so both effects are visible at once. The carriage has advanced 2.6 grid squares and the spokes have turned through -240.27° — two thirds of a turn, which is why the spoke pattern (10 spokes, 36° apart) does not repeat.

4.4 Rotation about a pivot that is not the origin

A rotation matrix rotates about the origin. To rotate about an arbitrary point $\mathbf{p}$ one conjugates:

$$\mathbf{R}_\mathbf{p}(\theta) = \mathbf{T}(\mathbf{p}) \mathbf{R}_z(\theta) \mathbf{T}(-\mathbf{p}). \quad (8)$$

In the renderer the $\mathbf{T}(-\mathbf{p})$ factor is absent, and this is deliberate rather than an omission: the barrel mesh is *authored* in a local frame whose origin already sits at the trunnion, so the geometry arrives pre-multiplied by $\mathbf{T}(-\mathbf{p})$ in the modelling stage. What the code writes,

```
glTranslatef(*PIVOT) # T(p)
glRotatef(self.elevation, 0, 0, 1) # Rz(theta)
self._draw_trunnions() # stays with the mount
glTranslatef(-self.recoil, 0, 0) # T(-rho) along the barrel's own axis
self._draw_barrel()
```

is (8) with the inverse translation folded into the model. Note where the trunnions are drawn: after the elevation rotation but *before* the recoil translation, so they swing with the barrel but do not slide with it, exactly as the real hardware behaves.

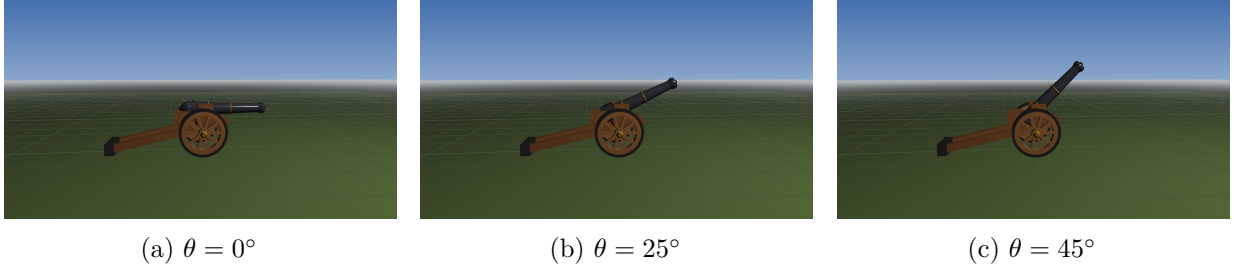


Figure 5: Elevation, clamped to $[0^\circ, 45^\circ]$. The pivot stays fixed in the carriage while the muzzle sweeps an arc of radius $\ell = 1.70$ m about it; the camera and light are identical in all three frames.

4.5 Where is the muzzle?

Firing needs the world position and direction of the muzzle, and guessing them with trigonometry written a second time is how the ball ends up leaving from inside the barrel. Instead `Cannon.barrel_matrix` rebuilds (5) in numpy — the same factors in the same order — and pushes two vectors through it:

$$\tilde{\mathbf{q}} = \mathbf{M}_{\text{barrel}}^{(\text{world})} \begin{pmatrix} \ell \\ 0 \\ 0 \\ 1 \end{pmatrix}, \quad \tilde{\mathbf{d}} = \mathbf{M}_{\text{barrel}}^{(\text{world})} \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix}, \quad \ell = 1.70 \text{ m.} \quad (9)$$

Expanding the product gives the closed form

$$\mathbf{q} = (x + p_x + (\ell - \rho) \cos \theta, \quad p_y + (\ell - \rho) \sin \theta, \quad 0), \quad \mathbf{d} = (\cos \theta, \sin \theta, 0), \quad (10)$$

and `verify.py` confirms the matrix chain and (10) agree to 0.000×10^0 m at $\theta \in \{0^\circ, 32^\circ, 45^\circ\}$ with $\rho \in \{0, 0.30\}$ m. Because $\tilde{\mathbf{d}}$ carries $w = 0$ it picks up the rotation but not the translations, so it is already the unit barrel axis ($|\mathbf{d}| = 1.000000$).

4.6 Recoil as a damped harmonic oscillator

On firing, the barrel is given a backwards impulse and then pulled home by an elastic mount. Writing $\rho(t) \geq 0$ for the distance the barrel has slid back along its own axis,

$$\ddot{\rho} + c \dot{\rho} + k \rho = 0, \quad \rho(0) = 0, \quad \dot{\rho}(0) = v_0, \quad (11)$$

with $k = 90.0 \text{ s}^{-2}$, $c = 2\zeta\sqrt{k} = 16.128 \text{ s}^{-1}$ and $v_0 = 3.40 \text{ m/s}$. The natural frequency, damping ratio and damped frequency are

$$\omega_n = \sqrt{k} = 9.4868 \text{ rad/s}, \quad \zeta = \frac{c}{2\sqrt{k}} = 0.850, \quad \omega_d = \omega_n \sqrt{1 - \zeta^2} = 4.9975 \text{ rad/s}, \quad (12)$$

so the system is *under*-damped ($\zeta < 1$) — chosen deliberately, since a critically damped return ($\zeta = 1$) creeps back and reads as sluggish, whereas a small overshoot looks mechanical. The solution of (11) is

$$\rho(t) = \frac{v_0}{\omega_d} e^{-\zeta\omega_n t} \sin(\omega_d t), \quad t_{\max} = \frac{1}{\omega_d} \arctan \frac{\omega_d}{\zeta\omega_n}, \quad (13)$$

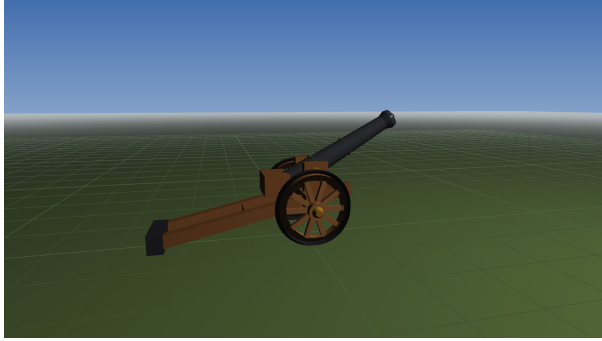
giving $t_{\max} = 0.1110 \text{ s}$ and $\rho_{\max} = 0.1464 \text{ m}$. The renderer does not evaluate (13); it integrates (11) with semi-implicit (symplectic) Euler,

$$\dot{\rho} \leftarrow \dot{\rho} + (-k\rho - c\dot{\rho})\Delta t, \quad \rho \leftarrow \rho + \dot{\rho}\Delta t, \quad (14)$$

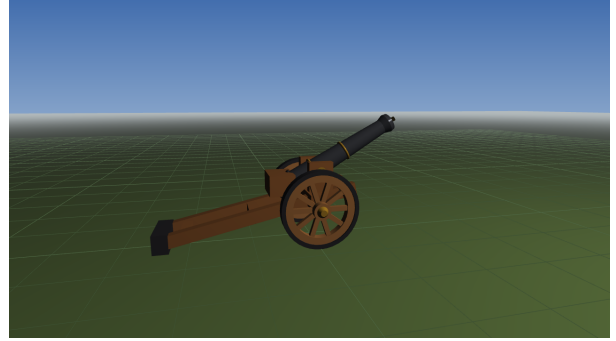
which stays stable at interactive time-steps. Measured against (13):

Δt	peak ρ (m)	t of peak (s)	error vs. (13)
1/240	0.1375	0.108	6.09 %
1/480	0.1419	0.110	3.05 %
1/4000	0.1459	0.111	0.37 %

Table 3: First-order convergence of the recoil integrator; the barrel settles to within 2 % of rest after ≈ 0.55 s in every case.



(a) at rest, $\rho = 0$



(b) $t = 0.11$ s after firing, near $\rho_{\max}$

Figure 6: Recoil. The barrel has slid back along its *own* axis — note that the breech has moved behind the cheeks while the trunnions and the whole carriage have not moved at all.

Because the translation $\mathbf{T}(-\rho, 0, 0)$ sits *inside* the elevation rotation, the world-space displacement is automatically along the elevated barrel: $\Delta \mathbf{q} = -\rho(\cos \theta, \sin \theta, 0)$. At $\theta = 25^\circ$ and $\rho = 0.30$ m, `verify.py` measures $(-0.2719, -0.1268, 0)$ m against a predicted $(-0.2719, -0.1268, 0)$ m.

4.7 Projectile motion

The ball leaves the muzzle at $\mathbf{q}$ (equation 10) with velocity $\mathbf{v}_0 = v_m \mathbf{d}$, $v_m = 15$ m/s, and thereafter feels only $\mathbf{g} = (0, -9.81, 0)$ m/s²:

$$\mathbf{p}(t) = \mathbf{q} + \mathbf{v}_0 t + \frac{1}{2} \mathbf{g} t^2. \quad (15)$$

Solving the vertical component for the height y_f at which the ball stops gives the flight time and the range

$$t_f = \frac{v_m \sin \theta + \sqrt{(v_m \sin \theta)^2 + 2g(q_y - y_f)}}{g}, \quad X = v_m \cos \theta t_f. \quad (16)$$

The HUD displays (16) live so the analytic prediction can be read off while the simulated ball is still in the air. The simulation itself uses the same semi-implicit Euler scheme as the recoil. For constant acceleration that scheme has an exactly known bias: after n steps of size Δt ,

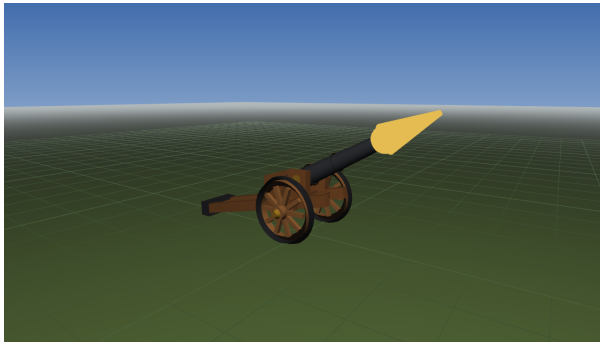
$$\mathbf{p}_n = \mathbf{q} + n\Delta t \mathbf{v}_0 + \mathbf{g} \frac{n(n+1)}{2} \Delta t^2 = \underbrace{\mathbf{q} + \mathbf{v}_0 t + \frac{1}{2} \mathbf{g} t^2}_{\text{exact}} + \frac{1}{2} \mathbf{g} \Delta t t, \quad (17)$$

i.e. the numerical ball sits $\frac{1}{2}g\Delta t t$ *below* the true parabola, so it lands slightly early and slightly short. At $\Delta t = 1/240$ s and $t \approx 1.8$ s that is about 3.7 cm, and indeed every simulated range below is a little under the analytic one:

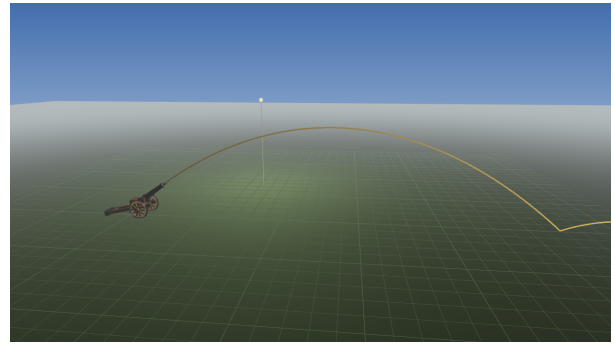
θ	t_f exact (s)	t_f sim (s)	X exact (m)	X sim (m)	rel. error
10°	0.8598	0.8583	12.7015	12.6794	0.174 %
20°	1.3090	1.3083	18.4506	18.4415	0.050 %
30°	1.7578	1.7542	22.8348	22.7873	0.208 %
35°	1.9717	1.9708	24.2265	24.2162	0.043 %
45°	2.3645	2.3625	25.0796	25.0581	0.086 %

Table 4: Simulated flight against the closed-form parabola, $\Delta t = 1/240$ s. Note that the optimum elevation is not 45° in general — it is 45° only for a launch and landing at the same height; firing from $q_y \approx 2$ m shifts the optimum a little below it.

On contact the ball is reflected with restitution $e = 0.35$ and tangential damping 0.72, $v_y \leftarrow -e v_y$, and is put to rest once $|v_y| < 0.8$ m/s, which stops the infinite bouncing that a constant-restitution model would otherwise produce.



(a) $t = 14$ ms: flash, ball leaving



(b) the completed parabola, $\theta = 35^\circ$

Figure 7: Firing. The trail is a poly-line of the ball's own history, so the shape in (b) *is* the integrated trajectory, not a drawn curve. Its span matches the 24.2 m predicted by (16).

4.8 Viewing and projection

Camera. The camera orbits a target $\mathbf{t}$ (the cannon, optionally offset down-range) at azimuth α , elevation β and distance r :

$$\mathbf{e} = \mathbf{t} + r(\cos \beta \cos \alpha, \sin \beta, \cos \beta \sin \alpha). \quad (18)$$

`gluLookAt` then builds an orthonormal eye basis from $\mathbf{e}$, $\mathbf{t}$ and the up hint $\mathbf{u}_h = (0, 1, 0)$:

$$\mathbf{f} = \frac{\mathbf{t} - \mathbf{e}}{\|\mathbf{t} - \mathbf{e}\|}, \quad \mathbf{s} = \frac{\mathbf{f} \times \mathbf{u}_h}{\|\mathbf{f} \times \mathbf{u}_h\|}, \quad \mathbf{u} = \mathbf{s} \times \mathbf{f}, \quad (19)$$

$$\mathbf{V} = \begin{pmatrix} s_x & s_y & s_z & 0 \\ u_x & u_y & u_z & 0 \\ -f_x & -f_y & -f_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \mathbf{T}(-\mathbf{e}). \quad (20)$$

The rows are the eye basis because the matrix is the *inverse* of the camera's placement: $\mathbf{V} = (\mathbf{R}\mathbf{T}(\mathbf{e}))^{-1} = \mathbf{T}(-\mathbf{e})\mathbf{R}^\top$ written out. The third row is negated because OpenGL's eye space looks down $-Z$. Pitch is clamped to $[-5^\circ, 85^\circ]$ so $\mathbf{f}$ never becomes parallel to $\mathbf{u}_h$, which would make $\mathbf{s}$ undefined.

Perspective. With a vertical field of view $\vartheta = 50^\circ$, aspect $a = w/h$, and near/far planes $n = 0.1$, $f = 200$, `gluPerspective` loads

$$\mathbf{P} = \begin{pmatrix} \frac{\cot(\vartheta/2)}{a} & 0 & 0 & 0 \\ 0 & \cot(\vartheta/2) & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \quad \cot(25^\circ) = 2.1445. \quad (21)$$

The -1 in the last row copies $-z_{\text{eye}}$ into w_{clip} ; the subsequent perspective divide by w is what makes distant objects small, and it is also what makes depth non-linear:

$$z_{\text{ndc}} = \frac{f+n}{n-f} + \frac{2fn}{(n-f)(-z_{\text{eye}})} \implies \text{half the depth buffer is spent inside } z_{\text{eye}} \in [-2n, -n]. \quad (22)$$

This is why $n = 0.1$ m and not 0.001 m: with the far plane at 200 m, a near plane a thousand times closer would crush the usable depth precision and produce z-fighting on the barrel bands, which are only 2 cm thick.

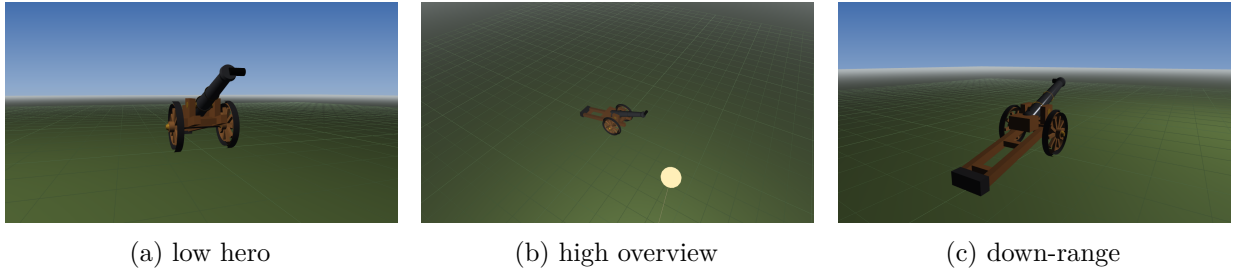


Figure 8: Three of the five camera presets (**C**). Perspective foreshortening is obvious in the grid: equal 1 m squares converge towards the horizon.

4.9 Normals

The normal matrix. A normal is not a direction that can be transformed by $\mathbf{M}$. If $\mathbf{t}$ is a surface tangent, then $\mathbf{N} \cdot \mathbf{t} = 0$ must still hold after transforming, i.e. $\mathbf{N}' \cdot (\mathbf{M}\mathbf{t}) = 0$. Substituting $\mathbf{N}' = \mathbf{A}\mathbf{N}$ gives $\mathbf{N}^\top \mathbf{A}^\top \mathbf{M} \mathbf{t} = 0$ for all $\mathbf{t}$, so $\mathbf{A}^\top \mathbf{M} = \mathbf{I}$ and

$$\mathbf{A} = (\mathbf{M}^{-1})^\top, \quad (23)$$

which is what `gl_NormalMatrix` contains (the upper-left 3×3 of the modelview, inverted and transposed). This project uses only translations and rotations, for which $(\mathbf{M}^{-1})^\top = \mathbf{M}$, so the distinction is invisible here — but it is the reason the shader is written with `gl_NormalMatrix` rather than `mat3(gl_ModelViewMatrix)`, and it would matter the moment a non-uniform `glScalef` were introduced.

Analytic normals for a truncated cone. Nearly every part of the cannon — barrel sections, muzzle swell, axle, hub, wheel rims — is a truncated cone, so its normal is worth deriving properly. Parameterise the lateral surface from radius r_0 at $z = 0$ to r_1 at $z = h$:

$$\mathbf{P}(\phi, t) = (r(t) \cos \phi, r(t) \sin \phi, ht), \quad r(t) = r_0 + (r_1 - r_0)t, \quad t \in [0, 1]. \quad (24)$$

The two tangents are $\partial \mathbf{P} / \partial \phi = (-r \sin \phi, r \cos \phi, 0)$ and $\partial \mathbf{P} / \partial t = ((r_1 - r_0) \cos \phi, (r_1 - r_0) \sin \phi, h)$, and their cross product is

$$\frac{\partial \mathbf{P}}{\partial \phi} \times \frac{\partial \mathbf{P}}{\partial t} = r(h \cos \phi, h \sin \phi, r_0 - r_1) \implies \boxed{\mathbf{N} = \frac{(h \cos \phi, h \sin \phi, r_0 - r_1)}{\sqrt{h^2 + (r_1 - r_0)^2}}} \quad (25)$$

The radius factor r divides out, so the normal depends only on ϕ — it is constant along each ruling, which is exactly why a cone shades with straight highlight bands. For a plain cylinder ($r_0 = r_1$) it collapses to the familiar $(\cos \phi, \sin \phi, 0)$; the naive guess of using that for a *tapered* section is wrong by $\arctan \frac{r_0 - r_1}{h}$, which for the barrel’s chase is 2.0° — small, but enough to misplace a tight specular highlight. Check 5 of `verify.py` confirms (25) is unit length to 2.2×10^{-16} and orthogonal to both tangents to 1.8×10^{-17} .

Spheres use $\mathbf{N} = \mathbf{P}/r$, and the torus (the brass astragal rings) uses $\mathbf{N} = (\cos \alpha \cos \beta, \sin \alpha \cos \beta, \sin \beta)$ for tube angle β around ring angle α .

4.10 The Phong reflection model

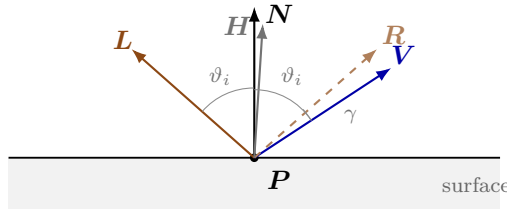


Figure 9: The vectors of the Phong model at a shading point. $\mathbf{R}$ is $\mathbf{L}$ mirrored about $\mathbf{N}$; the specular term falls off as $\cos^n \gamma$. Blinn’s variant replaces γ with the angle between $\mathbf{N}$ and the half-vector $\mathbf{H}$.

Each fragment’s colour is

$$\boxed{\mathbf{I} = \mathbf{m}_e + \mathbf{k}_a \otimes \mathbf{i}_a^{\text{glob}} + \frac{1}{k_c + k_l d + k_q d^2} \left[\mathbf{k}_a \otimes \mathbf{i}_a + \mathbf{k}_d \otimes \mathbf{i}_d \max(\mathbf{N} \cdot \mathbf{L}, 0) + \mathbf{k}_s \otimes \mathbf{i}_s \max(\mathbf{R} \cdot \mathbf{V}, 0)^n \right]} \quad (26)$$

where $\otimes$ is component-wise multiplication over the RGB channels, $d = \|\mathbf{P}_{\text{light}} - \mathbf{P}\|$, and

$$\mathbf{L} = \frac{\mathbf{P}_{\text{light}} - \mathbf{P}}{d}, \quad \mathbf{V} = \frac{-\mathbf{P}}{\|\mathbf{P}\|} \text{ (eye space)}, \quad \mathbf{R} = 2(\mathbf{N} \cdot \mathbf{L})\mathbf{N} - \mathbf{L}. \quad (27)$$

The three terms do three different jobs, and the project can show each in isolation:

Ambient

a constant floor standing in for all the light that has bounced off everything else. Pressing **L** kills the direct light and leaves only $\mathbf{m}_e + \mathbf{k}_a \otimes \mathbf{i}_a^{\text{glob}}$ — Figure 12(c). The scene becomes a flat silhouette, which is the point: ambient alone carries no shape information.

Diffuse Lambert’s cosine law, $\mathbf{N} \cdot \mathbf{L}$: it depends on where the light is but not on where the eye is, so it does not move when you orbit. This is what gives the carriage its solidity.

Specular

the mirror lobe, $(\mathbf{R} \cdot \mathbf{V})^n$: it depends on the eye, so it slides across the barrel as either the camera or the light moves (Figure 11). The exponent n controls tightness — $n = 8$ for wood spreads a weak sheen over the whole plank, $n = 96$ for iron concentrates it into the thin streak seen in Figure 10.

Attenuation and the light type. With $w = 1$ the light is a point source at a finite distance, $\mathbf{L}$ is recomputed per fragment, and the $1/(k_c + k_l d + k_q d^2)$ factor applies ($k_c = 1$, $k_l = 0.010$, $k_q = 0.0004$). Pressing **N** sets $w = 0$: the position becomes a direction, $\mathbf{L}$ is constant everywhere, and attenuation is disabled — the sun-at-infinity model. The difference is easiest to see on the large ground plane, which loses its pool of light and becomes uniformly lit.

Materials. Two of these are required to contrast; the project ships five.

Material	k_a	k_d	k_s	n
Oak (carriage)	(0.14, 0.09, 0.05)	(0.52, 0.32, 0.16)	(0.10, 0.08, 0.06)	8
Light oak (wheels)	(0.16, 0.11, 0.06)	(0.60, 0.40, 0.22)	(0.12, 0.10, 0.08)	10
Cast iron (barrel)	(0.06, 0.07, 0.08)	(0.19, 0.20, 0.23)	(0.92, 0.94, 1.00)	96
Brass (fittings)	(0.16, 0.12, 0.04)	(0.62, 0.47, 0.13)	(0.85, 0.75, 0.45)	64
Iron tyre	(0.05, 0.05, 0.06)	(0.13, 0.13, 0.15)	(0.55, 0.55, 0.60)	48
Field (ground)	(0.10, 0.13, 0.08)	(0.34, 0.44, 0.24)	(0.04, 0.05, 0.04)	4

Table 5: Phong coefficients. Wood and iron differ by roughly $9\times$ in specular strength and $12\times$ in exponent — that ratio, not the base colour, is what makes one look like timber and the other like metal.

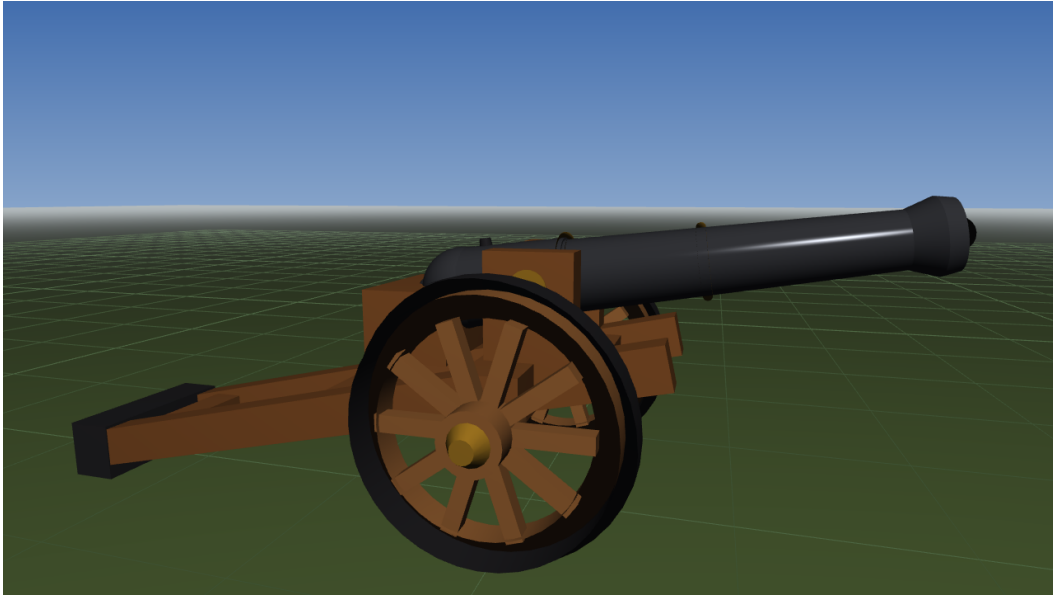


Figure 10: Two materials under one light. The oak beams and spokes show a broad, almost invisible sheen ($k_s \approx 0.1$, $n = 8$) and read as matte; the barrel carries a narrow white streak ($k_s \approx 0.95$, $n = 96$) and reads as polished iron. The brass hub cap sits between the two. Nothing here is a painted highlight — all of it comes out of (26).

Why the highlight needs the light in the right place. A specular streak appears where the mirror condition $\mathbf{R} = \mathbf{V}$ is met, equivalently where $\mathbf{L} + \mathbf{V} \parallel \mathbf{N}$. On a cylinder of axis $\hat{\mathbf{a}}$ every normal satisfies $\mathbf{N} \cdot \hat{\mathbf{a}} = 0$, so a *perfect* highlight exists only if

$$(\mathbf{L} + \mathbf{V}) \cdot \hat{\mathbf{a}} = 0, \quad (28)$$

that is, only if the light and the eye lie on opposite sides of the plane through the barrel perpendicular to its axis. When they lie on the same side, $\max_{\phi}(\mathbf{N} \cdot \mathbf{H})$ can be as low as 0.89, and raised to $n = 96$ that is 1.4×10^{-5} — effectively black. This is not a defect; it is (26) behaving correctly, and it is why the light is placed at azimuth 122° against a camera at 58° for Figure 12.

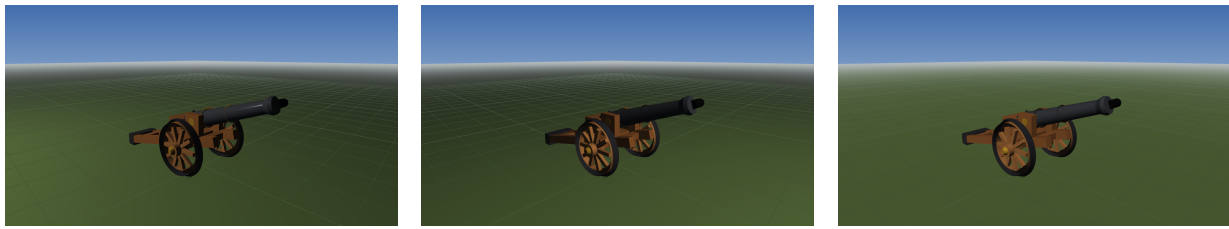
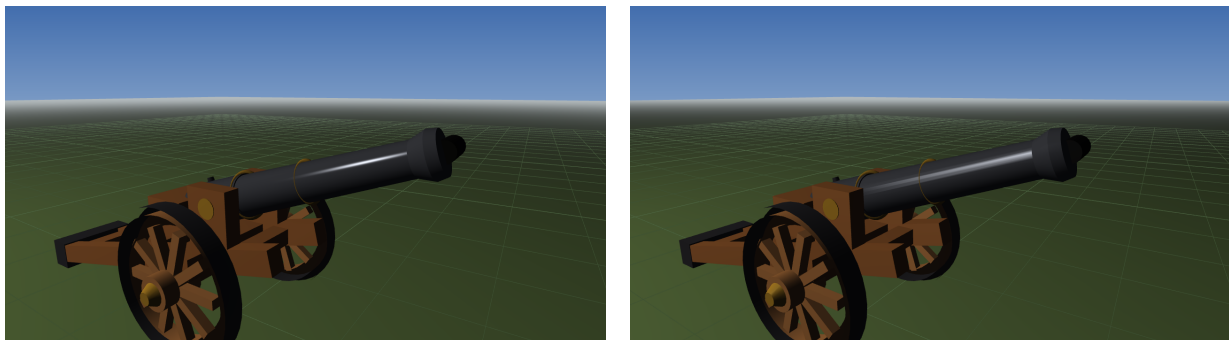
(a) light at 120° (b) light at 10° (c) directional ($w = 0$)

Figure 11: Identical camera, identical materials, moved light. In (a) condition (28) is satisfied and the barrel carries a bright streak; in (b) the light has swung to the eye's own side and the streak vanishes while the diffuse shading merely shifts. In (c) the same light is made directional: the falloff across the ground disappears.

4.11 Gouraud versus Phong shading

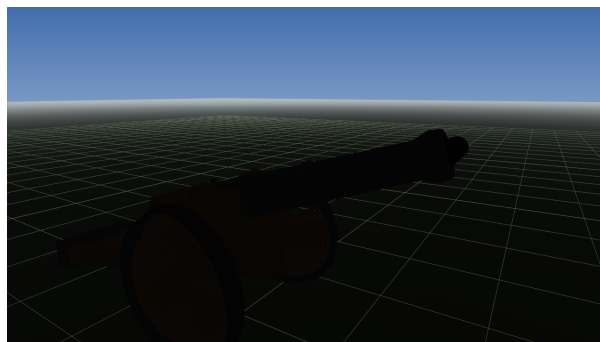
The reflection model of §4.10 says *what* to compute; shading says *where*. The fixed-function pipeline evaluates (26) once per vertex and interpolates the resulting colour across the triangle (Gouraud shading). The GLSL path interpolates the *normal* and evaluates (26) once per fragment (Phong shading). Since a specular lobe of exponent 96 is far narrower than a triangle, per-vertex evaluation either misses the highlight entirely or smears it into a polygonal blob.

Pressing **F** switches paths at runtime. Crucially, both paths read the *same* `glMaterial/glLight` state — the shader uses the legacy `gl_FrontMaterial` and `gl_LightSource[0]` built-ins — so the comparison isolates the shading frequency and nothing else.



(a) per-fragment Phong

(b) per-vertex Gouraud



(c) light off: the ambient term alone

Figure 12: The same frame under the two shading paths, and with the direct light disabled. Measured over the whole frame, (a) and (b) differ by more than $4/255$ on 8.82% of pixels, with a maximum channel difference of $88/255$; peak luminance is 255 under Phong against 205 under Gouraud, and the Gouraud highlight is visibly banded along the 40 slices of the tube.

4.12 Atmospheric fog

The ground plane is finite (100×100 m), and its edge would otherwise be a hard line across the sky. Both paths therefore apply exponential-squared fog in eye space,

$$f = e^{-(\varrho z_{\text{eye}})^2}, \quad C = f C_{\text{lit}} + (1 - f) C_{\text{fog}}, \quad \varrho = 0.020 \text{ m}^{-1}, \quad (29)$$

and the lowest stop of the sky gradient is set to C_{fog} , so the plane dissolves into the horizon. At the 3–9 m working distance of the cannon $f > 0.99$, so the model itself is essentially unfogged.

5 Implementation notes

5.1 The shader

```

varying vec3 vNormalEye; // interpolated normal -> Phong (not Gouraud) shading
varying vec3 vPosEye;
uniform bool uLightEnabled, uBlinn;
uniform vec3 uFogColor; uniform float uFogDensity;

void main() {
    vec3 N = normalize(vNormalEye);
    vec3 color = gl_FrontLightModelProduct.sceneColor.rgb; // emission + k_a * global ambient

    if (uLightEnabled) {
        vec4 lpos = gl_LightSource[0].position; // already in eye space
        vec3 Lv = lpos.xyz - vPosEye * lpos.w; // w = 0 -> directional
        float d = length(Lv);
        vec3 L = Lv / max(d, 1e-6);

        float att = 1.0;
        if (lpos.w > 0.5)
            att = 1.0 / (gl_LightSource[0].constantAttenuation
                        + gl_LightSource[0].linearAttenuation * d
                        + gl_LightSource[0].quadraticAttenuation * d * d);

        vec3 V = normalize(-vPosEye); // local viewer
        float ndl = max(dot(N, L), 0.0);
        float spec = 0.0;
        if (ndl > 0.0) {
            if (uBlinn) { vec3 H = normalize(L + V);
                spec = pow(max(dot(N, H), 0.0), gl_FrontMaterial.shininess); }
            else { vec3 R = reflect(-L, N);
                spec = pow(max(dot(R, V), 0.0), gl_FrontMaterial.shininess); }
        }
        color += att * (gl_LightSource[0].ambient.rgb * gl_FrontMaterial.ambient.rgb
                      + gl_LightSource[0].diffuse.rgb * gl_FrontMaterial.diffuse.rgb * ndl
                      + gl_LightSource[0].specular.rgb * gl_FrontMaterial.specular.rgb * spec);
    }
    float z = length(vPosEye);
    color = mix(uFogColor, color, clamp(exp(-(uFogDensity*z)*(uFogDensity*z)), 0.0, 1.0));
    gl_FragColor = vec4(color, gl_FrontMaterial.diffuse.a);
}

```

Listing 1: The fragment shader — equation (26), verbatim.

If `glCompileShader` or `glLinkProgram` fails on any driver, the exception is caught, `PhongShader.available` stays `False`, and the program runs entirely on the fixed-function pipeline. Nothing else in the code has to know which path is live.

Blinn–Phong. Pressing **B** swaps $(\mathbf{R} \cdot \mathbf{V})^n$ for $(\mathbf{N} \cdot \mathbf{H})^n$ with $\mathbf{H} = \widehat{\mathbf{L} + \mathbf{V}}$. Since the angle between $\mathbf{N}$ and $\mathbf{H}$ is about half the angle between $\mathbf{R}$ and $\mathbf{V}$, the Blinn lobe is roughly twice as wide for the same n ; matching the two requires $n_{\text{Blinn}} \approx 4n_{\text{Phong}}$.

5.2 Drawing unlit geometry inside a lit frame

The projectile trails, the light marker and the HUD are flat-coloured. Simply calling `glDisable(GL_LIGHTING)` is not enough when a shader is bound, because the shader ignores that state entirely and would keep shading them with the last material — an early version of this project drew the trails almost black for exactly that reason. `lighting.unlit` is a context manager that drops out of both the shader and fixed-function lighting and restores whichever was active:

```
with unlit(): # leaves the program, disables GL_LIGHTING
    glBegin(GL_LINE_STRIP)
    ... # glColor3f now actually reaches the framebuffer
    glEnd()
```

5.3 Geometry budget

Part	Primitive	Tessellation
Trail beams, transoms, cheeks, spokes	box	6 quads each
Barrel (breech, reinforce, chase, swell, bore)	truncated cones + sphere	40 slices
Wheel tyre and felloe	tube (annulus)	40 slices
Wheel hub, axle, trunnions	cylinders	16–24 slices
Astragal rings	torus	32×12
Cannonball	UV sphere	20×14
Ground	tessellated quad grid	150×150

Table 6: The ground is finely tessellated on purpose: under the fixed-function per-vertex path a single large quad would sample the point light at only four corners and the pool of light would vanish.

6 Results

The scene runs at the display refresh rate on the test machine; the HUD in Figure 13 reports a measured frame rate obtained by timing 30 consecutive `render()` calls with `glFinish` barriers, not an estimate.

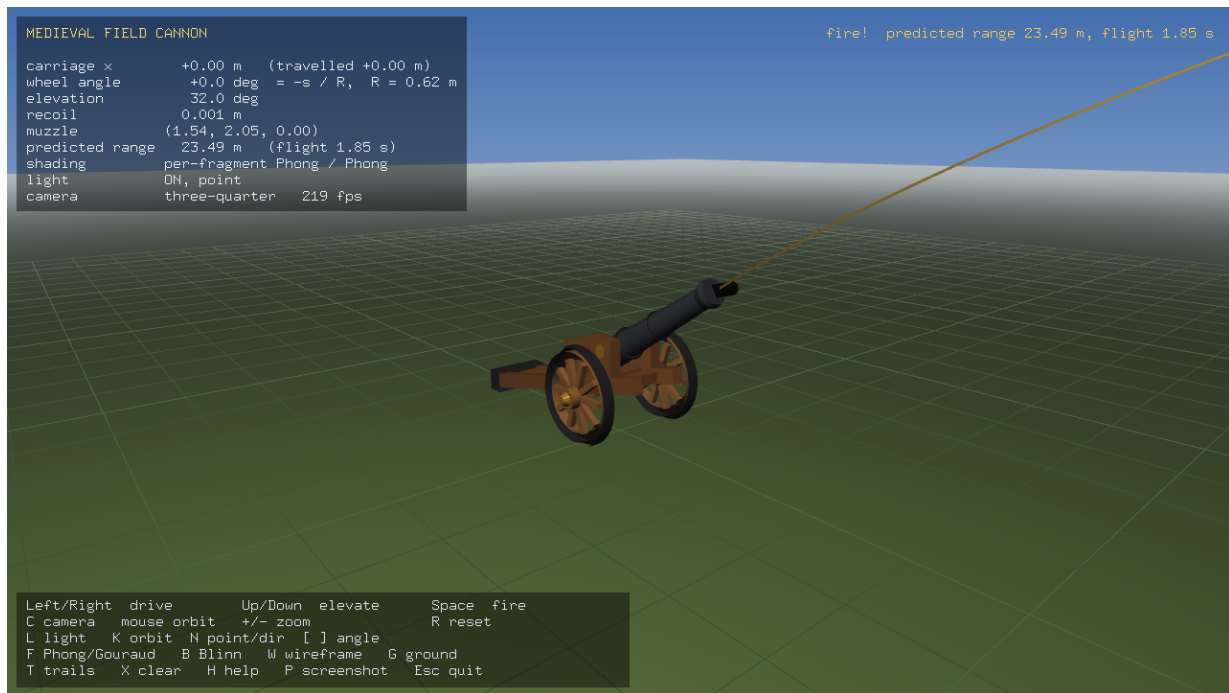


Figure 13: The live HUD. It reports the carriage position and distance travelled, the wheel angle beside the $-s/R$ rule that produced it, the elevation, the instantaneous recoil, the muzzle point from (10), and the analytic range prediction (16) for the shot currently in the air.

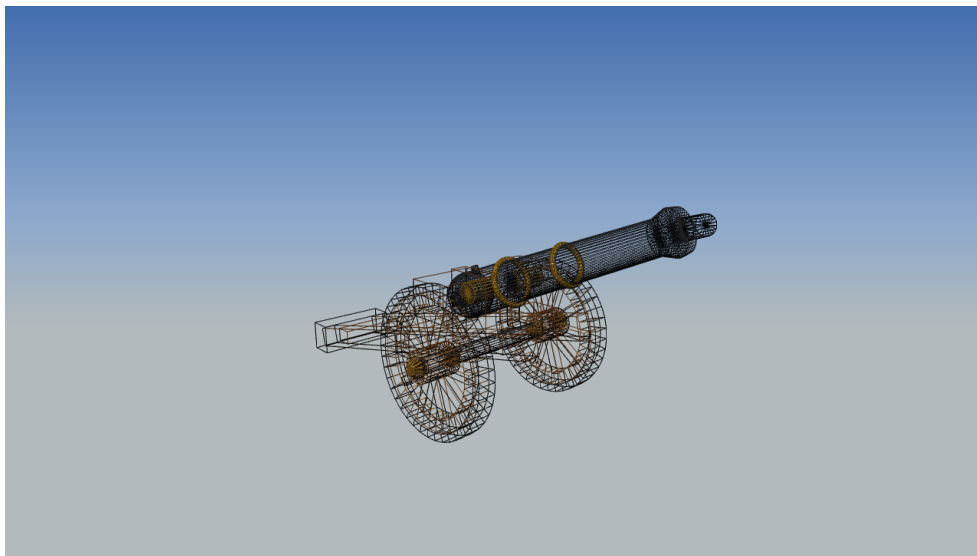


Figure 14: Wireframe (**W**). The structure is visible: 10 box spokes per wheel, the annular tyre and felloe, the stack of truncated cones that forms the barrel, and the sphere at the breech. No external model files are used — every vertex is generated by `utils.py`.

7 Limitations and possible extensions

- **No shadows.** The cannon does not cast one, which is the most noticeable absence. A planar projection shadow onto $y = 0$ would be a single extra matrix and would cost almost nothing.
- **One light.** The rig supports `GL_MAX_LIGHTS = 8`; the shader hard-codes `gl_LightSource[0]` and would need a loop.

- **Legacy GLSL.** Using the deprecated built-ins keeps the shader and fixed-function paths in perfect agreement — ideal for the comparison in Figure 12 — but a modern core-profile port would have to pass materials and lights as explicit uniforms.
- **Flat ground, no drag.** The projectile ignores air resistance and the terrain is a plane; both are deliberate, per the brief’s instruction to keep the scope tight.
- **No textures.** All surface variety comes from the material coefficients in Table 5, which is what makes the lighting model so legible in the figures.

A Verification output

Produced by `python verify.py`; every number quoted in §4 comes from this run.

```
Medieval Field Cannon -- numerical verification

-----
1. Rolling constraint phi = -s / R
-----

wheel radius R : 0.6200 m
distance driven s : 2.6000 m
expected -s/R : -240.2726 deg
simulated wheel_angle : -240.2726 deg
error : 2.672e-12 deg
rim arc length R|phi| : 2.6000 m (must equal s)

-----
2. Muzzle point from the composite matrix
-----

elev 0.0 deg, recoil 0.00 m
matrix chain : ( 3.3000, 1.1500, 0.000)
closed form : ( 3.3000, 1.1500, 0.000)
|difference| : 0.000e+00 m
axis direction : (1.0000, 0.0000, 0.0000) |d| = 1.000000
elev 32.0 deg, recoil 0.00 m
matrix chain : ( 3.0417, 2.0509, 0.000)
closed form : ( 3.0417, 2.0509, 0.000)
|difference| : 0.000e+00 m
axis direction : (0.8480, 0.5299, 0.0000) |d| = 1.000000
elev 45.0 deg, recoil 0.30 m
matrix chain : ( 2.5899, 2.1399, 0.000)
closed form : ( 2.5899, 2.1399, 0.000)
|difference| : 0.000e+00 m
axis direction : (0.7071, 0.7071, 0.0000) |d| = 1.000000

-----
3. Projectile: semi-implicit Euler vs. the closed-form parabola
-----

muzzle speed v0 : 15.00 m/s
gravity g : 9.81 m/s^2

elev t_exact t_sim R_exact R_sim rel.err
10.0 0.8598 0.8583 12.7015 12.6794 0.174 %
20.0 1.3090 1.3083 18.4506 18.4415 0.050 %
30.0 1.7578 1.7542 22.8348 22.7873 0.208 %
35.0 1.9717 1.9708 24.2265 24.2162 0.043 %
45.0 2.3645 2.3625 25.0796 25.0581 0.086 %

HUD prediction helper (fired from the muzzle height, y = 0 target):
elev 32 deg -> flight 1.847 s, range 23.494 m (muzzle height 2.051 m)

-----
4. Recoil: damped harmonic oscillator
-----

k = 90.00 1/s^2 c = 16.1276 1/s
natural freq w_n = sqrt(k) : 9.4868 rad/s (1.510 Hz)
damping ratio zeta = c / 2 sqrt(k) : 0.8500 (under-damped)
damped freq w_d = w_n sqrt(1-z^2) : 4.9975 rad/s
impulse v0 : 3.40 m/s
analytic peak time t_p : 0.1110 s
analytic peak x(t_p) : 0.1464 m
dt = 1/240 peak 0.1375 m at t = 0.108 s, |err| vs analytic 6.09 %, settles (2%) at t = 0.558 s
dt = 1/480 peak 0.1419 m at t = 0.110 s, |err| vs analytic 3.05 %, settles (2%) at t = 0.556 s
dt = 1/4000 peak 0.1459 m at t = 0.111 s, |err| vs analytic 0.37 %, settles (2%) at t = 0.553 s
clamp RECOIL_MAX : 0.4200 m (never reached here)

-----
5. Analytic normals of the truncated cone are correct
-----

surface P(phi,t) = ((r0+(r1-r0)t)cos phi, ... , h t), r0=0.175 r1=0.125 h=1.420
N = normalize(h cos phi, h sin phi, r0-r1)
max | |N| - 1 | : 2.220e-16
max |N . tangent| : 1.841e-17 (0 => N is the true surface normal)

-----
6. Hierarchy: a child inherits every ancestor transform
-----

muzzle at x=0.0 : (1.6407, 1.8685, 0.000)
muzzle at x=4.0 : (5.6407, 1.8685, 0.000)
delta : (4.0000, 0.0000, 0.000) <- pure translation of the root
```

```
recoil 0.30 m : delta = (-0.2719, -0.1268, 0.000)
expected -rho*(cos25, sin25, 0) = (-0.2719, -0.1268, 0.000)
|error| : 1.241e-16 m
```

All checks completed.